# Honeypot IDS – Pool test report

Generated: 2026-10-05 09:38:50
Target: ids (local)  (https://localhost)
Exploit sent from each window: CVE-2023-28121 — WooCommerce Payments admin bypass
Windows: 3

---

## Result

INCOMPLETE **Not every frame reached a honeypot pool**

- Session not bound to a pool: Attacker A, Attacker B, Attacker C (the attack has not pushed its score over the diversion threshold, or has not been sent).

## What is being tested

Attackers are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees three separate attackers. By default every malicious session is given a honeypot pool **of its own**: the router hands a new attacker a free ready pool no other session owns, and pool_manager builds a spare after each assignment. Two attackers only share a pool when none is free – growth is capped because the host is short of memory, CPU or disk, or every ready pool is already owned – and the router then reuses the ready pools in strict round-robin order.

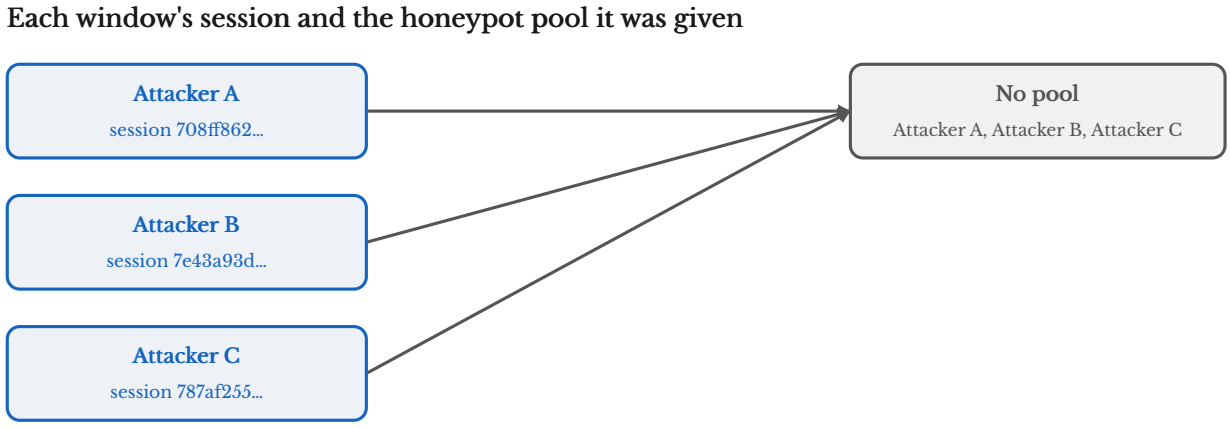



**Figure 1.** One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

## Windows

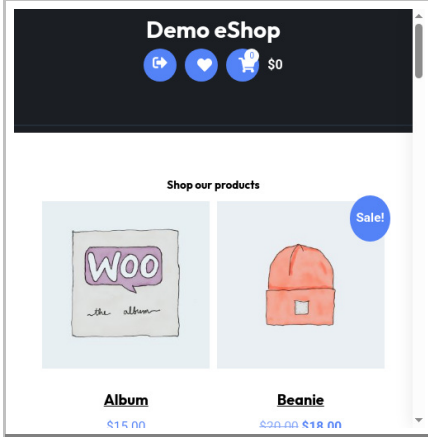
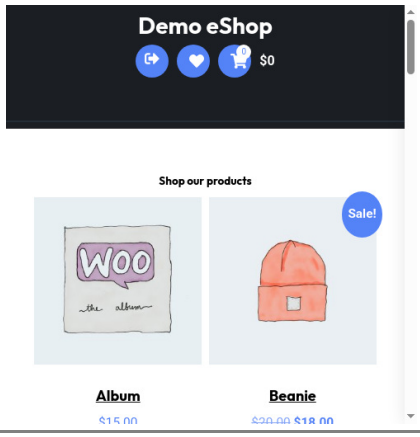
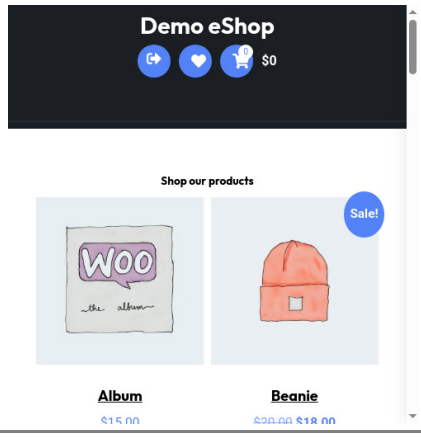
| Window | Session | Honeypot pool | Language | User-Agent (tail) |
|---|---|---|---|---|
| Attacker A | 708ff862... | none | en-US,en;q=0.9 | 6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A |
| Attacker B | 7e43a93d... | none | de-DE,de;q=0.9 | 6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/B |
| Attacker C | 787af255... | none | fr-FR,fr;q=0.9 | 6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C |

# Step by step

All three windows move in every step. An attacker's pool must appear in their own step, differ from the earlier ones, and never change afterwards.

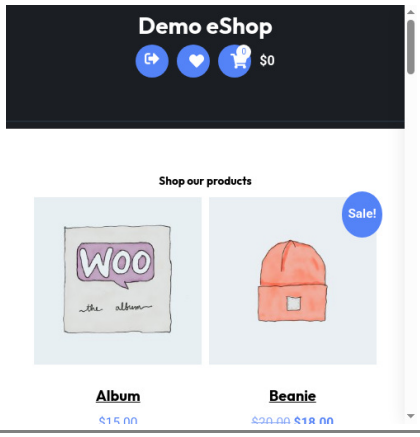
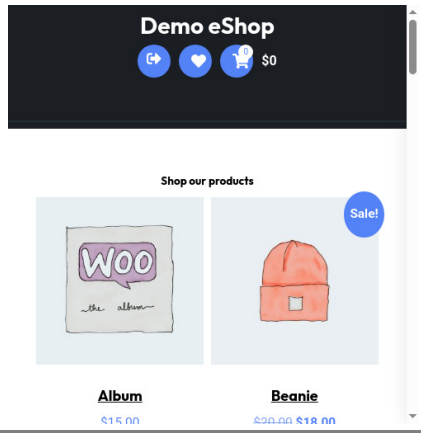
## Step 1: All windows open the shop

Three new sessions, all routed to production.

| Attacker A | Attacker B | Attacker C |
|---|---|---|
| Browses the shop | Browses the shop | Browses the shop |
| PRODUCTION<br>no pool • score 0 | PRODUCTION<br>no pool • score 0 | PRODUCTION<br>no pool • score 0 |
|  |  |  |

## Step 2: Attacker A attacks

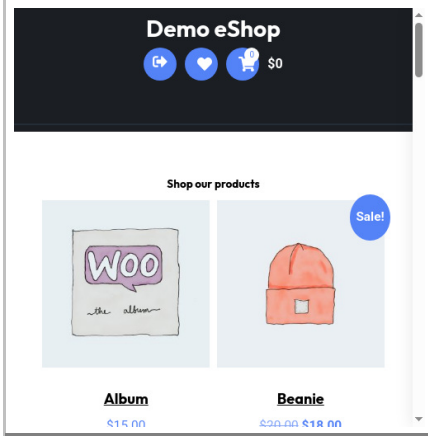
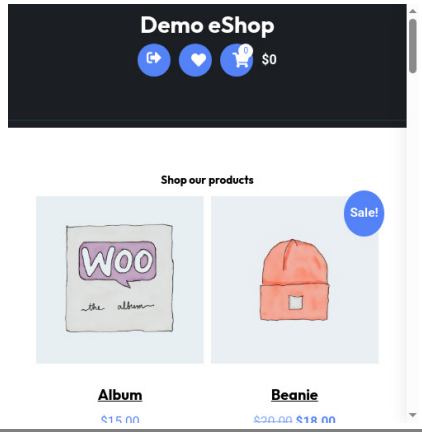
A's session is bound to a honeypot pool; B and C stay on production.

| Attacker A | Attacker B | Attacker C |
|---|---|---|
| Opens the exploit | Browses the shop | Browses the shop |
| PRODUCTION<br>no pool • score 25 | PRODUCTION<br>no pool • score 0 | PRODUCTION<br>no pool • score 0 |
|  |  |  |

## Step 3: Attacker B attacks

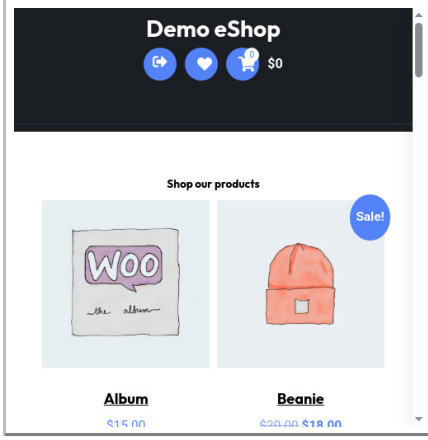
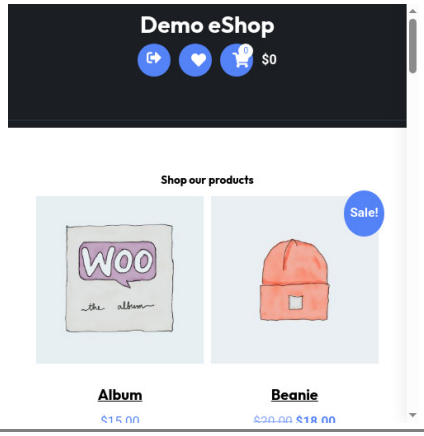
B must get a different pool from A; A keeps its pool.

| Attacker A | Attacker B | Attacker C |
|---|---|---|
| Browses the shop | Opens the exploit | Browses the shop |
| PRODUCTION | PRODUCTION | PRODUCTION |

| | | |
|---|---|---|
| no pool • score 25 | no pool • score 25 | no pool • score 0 |
|  |  |  |

## Step 4: Attacker C attacks

C must get a pool of its own as well.

| Attacker A | Attacker B | Attacker C |
|---|---|---|
| Browses the shop | Browses the shop | Opens the exploit |
| PRODUCTION<br>no pool • score 25 | PRODUCTION<br>no pool • score 25 | PRODUCTION<br>no pool • score 25 |
|  |  |  |

## Step 5: All three attack at once

Sticky binding: nobody changes pool, nobody is merged.

| Attacker A | Attacker B | Attacker C |
|---|---|---|
| Opens the exploit | Opens the exploit | Opens the exploit |
| PRODUCTION<br>no pool • score 50 | PRODUCTION<br>no pool • score 50 | PRODUCTION<br>no pool • score 50 |




# Pool registry at export time

**Growing normally** – the host has room, so every new attacker gets a pool of their own and a spare is built after each assignment.
Ready pools: 3 (1, 2, 3). Free (unowned) spares: 3. Attackers per pool: pool 1: 0, pool 2: 0, pool 3: 0. Round-robin reuse assignments so far: 0.